# 프로그래밍I - 관계기반

Chap #0 – 수업 및 평가 안내

# 강의 개요 및 클래스룸 가입

[강의 개요]

- 교재: 문제해결을 위한 창의적 알고리즘(중급/고급) – 클래스룸 파일 업로드
- 지도 교사: 이산 T(중급-탐색기반), 김정화T(고급-관계기반)
- 학점: 3학점(주당 3시간: 2+1 진행)
  - 1차 지필 전: 이산T(중급-1시간), 김정화T(고급-2시간)
  - 1차 지필 후: 이산T(중급-2시간), 김정화T(고급-1시간)

[클래스룸 가입]

- **classroom.google.com 접속 후 수업 참여**
- 수업코드: hvf4yrig
- **수업과 관련된 모든 자료는 클래스룸에 업로드 예정**

# 평가 안내(안)

## [변경 시 다시 안내합니다!]

| 구분 | 과목만점(100점) | | | | | | |
|---|---|---|---|---|---|---|---|
| | 지필평가(30%) | 수행평가(70%) | | | | | |
| | 1회고사 | 탐색기반 연습문제 | 관계기반 연습문제 | 탐색기반 문제해결 1 | 관계기반 문제해결 1 | 탐색기반 문제해결 2 | 관계기반 문제해결 2 |
| 평가방법 | 서답형 | Online judge 및 classroom을 활용한 프로그래밍 문제 해결력 연습문제 풀이 평가 | | Online judge를 활용한 프로그래밍 문제 해결력 실기 평가 | | | |
| 배점 | 100점 | 15점 | 15점 | 10점 | 10점 | 10점 | 10점 |
| 반영비율 | 30% | 15% | 15% | 10% | 10% | 10% | 10% |

| 성취 수준 | 점수(점) |
|---|---|
| 연습문제 해결비율이 100%일 때 | 15 |
| 연습문제 해결비율이 90% 이상 100% 미만일 때 | 14 |
| 연습문제 해결비율이 80% 이상 90% 미만일 때 | 13 |
| 연습문제 해결비율이 70% 이상 80% 미만일 때 | 12 |
| 연습문제 해결비율이 60% 이상 70% 미만일 때 | 11 |
| 연습문제 해결비율이 50% 이상 60% 미만일 때 | 10 |
| 연습문제 해결비율이 40% 이상 50% 미만일 때 | 9 |
| 연습문제 해결비율이 30% 이상 40% 미만일 때 | 8 |
| 연습문제 해결비율이 20% 이상 30% 미만일 때 | 7 |
| 연습문제 해결비율이 10% 이상 20% 미만일 때 | 6 |
| 연습문제 해결비율이 10% 미만일 때(단, 제출한 경우) | 5 |
| 모든 연습문제 미제출 시 | 3 |
| 부정행위 적발 시(타인의 소스코드를 복사하여 제출하는 등 정당하지 않은 방법으로 제출한 사실이 적발되었을 경우) | 0 |

| 성취 수준 | 점수(점) |
|---|---|
| 제시된 문제를 6개 이상 해결하였을 때 | 10 |
| 제시된 문제를 5개 해결하였을 때 | 9 |
| 제시된 문제를 4개 해결하였을 때 | 8 |
| 제시된 문제를 3개 해결하였을 때 | 7 |
| 제시된 문제를 2개 해결하였을 때 | 6 |
| 제시된 문제를 1개 해결하였을 때 | 5 |
| 코드를 제출하였으나 제시된 문제를 하나도 해결하지 못한 경우 | 4 |
| 문제 해결을 위한 코드를 전혀 제출하지 않은 경우 | 2 |
| 부정행위 적발 및 퀴즈에 무단 불참 시 | 0 |

# 수업 참여 시 주의사항




- 수업 시작 전 입실 완료!
- 강의실 음식물 섭취 금지!
- 수업과 무관한 전자기기 사용 금지!
- 퇴실 전 반드시 자리 정리!
- 과제는 가능한 스스로 해결!

# 프로그램 실행시간 측정

[코딩해 봅시다!]

- 1~입력된 숫자(int 범위)까지의 합을 구하는 시간은?
- 헤더 파일 추가: `<time.h>`
- 시작 시간 측정: `time_t start=clock();`
- 종료 시간 측정: `time_t end=clock();`
- 측정 시간 출력(초):

```
printf("%f", (float)(end-start)/CLOCKS_PER_SEC);
```

# <코드 예시>

```
#include <stdio.h>
#include <time.h>

int main() {
    int n;    long long sum=0;
    scanf("%d", &n);
    time_t start=clock();   //측정 시작
    for(int i=1; i<=n; i++)    sum +=i;
    printf("%lld\n", sum);
    time_t end=clock();     //측정 종료
    printf("%f", (float)(end-start)/CLOCKS_PER_SEC);  //초 단위 출력
}
```

# 프로그래밍I - 관계기반

Chap #1 – 수학적 귀납법과 점화식

# 탐색기반 설계 VS 관계기반 설계

[**탐색기반** 설계]

- 문제 해결에 필요한 데이터들을 탐색 가능한 형태로 구조화한 후 (컴퓨터의 빠른 연산 속도를 이용하여) 해 집합을 탐색하여 해를 구하는 기법
- 수학적 배제, 경험적 배제 등을 통해 해 집합의 크기를 줄일 수 있음

[**관계기반** 설계]

- 문제의 정의 및 상태를 함수로 표현하고, 이 함수들의 관계를 이용하여 해를 구하는 기법
- 함수들 간의 관계를 점화식 또는 이와 유사한 형태로 표현
- 점화식과 수학적 귀납법을 기반으로 문제 해결

# 수학적 귀납법

[수학적 귀납법]

자연수 n에 대해 명제 P(n)이 모든 자연수 n에 대해 성립함을 증명하기 위한 수학의 증명법 중 한 방법

1. P(1)이 성립함을 보인다.
2. P(k)가 성립한다고 가정하고 P(k+1)이 성립함을 보인다.

[수학적 귀납법을 이용한 문제해결 절차]

1. 입력 값이 n인 문제의 해를 f(n)으로 정의한다.
2. f(1)을 직접 구한다.
3. f(k)를 이미 구했다고 가정하고, f(k)를 통해 f(n)을 구한다.
   - k는 다양한 형태(예: n-1, n/2 등)가 될 수 있으며, 문제에 따라 적절한 형태로 설계해야 함

# 귀납법을 이용한 재귀함수 설계

[예제1]

- 임의의 정수 n을 입력 받아 1부터 n까지 합을 구하는 프로그램을 작성하시오.
- 교재 p.9~12

[방법1: k = n-1로 정의]

1. 함수 정의: f(n)=1부터 n까지의 합
2. f(1)=1
3. f(n-1)을 구했다 가정하면, f(n)=f(n-1)+n

예> f(10)=1+2+3+4+5+6+7+8+9+10
f(9) =1+2+3+4+5+6+7+8+9
∴ f(10) = f(9) +10
☞일반화하면, f(n)=f(n-1)+n

# 귀납법을 이용한 재귀함수 설계

[방법2: k = $\frac{n}{2}$ 으로 정의]

1. n이 짝수 일 때

예> f(10) = 1+2+3+4+5+6+7+8+9+10

= f(5) +(5+1)+(5+2)+(5+3)+(5+4)+(5+5)

= f(5) + f(5) + 5X5

= 2f(5) +$5^2$

일반화 하면, f(n) = $2f\left(\frac{n}{2}\right) + \left(\frac{n}{2}\right)^2$

# 귀납법을 이용한 재귀함수 설계

[방법2: k = $\frac{n}{2}$ 으로 정의]

2. n이 홀수 일 때

예> f(11) = 1+2+3+4+5+6+7+8+9+10+11

= f(5) +(5+1)+(5+2)+(5+3)+(5+4)+(5+5)+11

= 2f(5) + 5*5 +11

일반화 하면, $f(n) = 2f\left(\frac{n-1}{2}\right) + \left(\frac{n-1}{2}\right)^2 + n = 2f\left(\frac{n-1}{2}\right) + \frac{n^2-2n+1+4n}{4}$

$= 2f\left(\frac{n-1}{2}\right) + \left(\frac{n+1}{2}\right)^2 = 2f\left(\left[\frac{n}{2}\right]\right) + \left(\frac{n+1}{2}\right)^2$

# 귀납법을 이용한 재귀함수 설계

[방법2: k = $\frac{n}{2}$ 으로 정의]

3. n이 홀수와 짝수 상관없이 쓸 수 있도록 일반화하면,

예> 짝수일 때: $f(n) = 2f\left(\frac{n}{2}\right) + \left(\frac{n}{2}\right)^2$

홀수일 때: $f(n) = 2f\left(\left[\frac{n}{2}\right]\right) + \left(\frac{n+1}{2}\right)^2$

☞ $f(n) = 2f\left(\left[\frac{n}{2}\right]\right) + \left(\left[\frac{n+1}{2}\right]\right)^2$

코드 적용 시 : `f(n) = 2f(n/2) + ((n+1)/2)²`

# 실습 #1

- 방법1과 방법2를 이용하여 1부터 n까지의 합을 구하는 프로그램을 작성하고, 그 성능을 비교해 보시오.

| n | 1,000 | 10,000 | 100,000 | 1,000,000 | 10,000,000 | 100,000,000 |
|---|---|---|---|---|---|---|
| 반복문 | | | | | | |
| 재귀(n-1) | | | | | | |
| 재귀(n/2) | | | | | | |

# 귀납법을 이용한 재귀함수 설계

[예제2]

- 임의의 10진법 정수 n을 입력 받아 이를 k진법으로 출력하는 프로그램을 작성하시오(단, 0<=n<=100,000; 2<=k<=20).
- 교재 p.12~17

1. 함수 정의

f(n, k) : 10진수 n을 k 진법으로 출력

2. 바로 출력 가능한 값:

f(n, k) = n출력 (단 , n<k) 혹은

// f(n, k) = 0 or 출력 안 함 (단, n=0)

3. 나머지 경우:

f(n, k) = f(n/k, k) 출력 후, n%k 출력

정리하면,

$$f(n,k) = \begin{cases} print(n) & (n<k) \\ f(n/k,\ k),\ print(n\%k) & (n>=k) \end{cases}$$

매개 변수의 개수는 문제 상황에 맞게 조정 가능

예>

1 2

2

---

6 … 0 ④

2

---

3 … 0 ③

2

---

1 … 1 ②

①

k의 범위를 다시 살펴 보면!!

# 문제1 – 숫자 뒤집기(S)

[문제 설명]

- 하나의 정수(n)를 입력 받아 거꾸로 출력하는 프로그램을 작성하시오.

  단, 유효 숫자만 출력  (1 <= n <= 50,000)

  예1>  123 => 321

  예2>  12300 => 321

- 교재 p.18~23

# 숫자 뒤집기 – 해결 전략 #1

**[#1: 나머지 연산자(%)와 flag를 이용]**

1. f($n$) 정의 : $n$ 을 거꾸로 출력한 수
2. $n$ 이 $0$이면 return
3. 그렇지 않으면, $n\%10$ 을 출력한 후 f($[n/10]$)을 호출

But! 맨 앞 $0$이 출력되는 문제 발생

=> flag 를 사용하여 맨 앞의 출력이 $0$인 경우 출력하지 않도록 처리

# 연습문제를 풀어서 제출하세요!

[SASA-OJ]

- [[대회]-[2026_2학기_프로그래밍I] 관계기반설계 연습문제 #1

- 기한 내에 제출하세요!
- 수업내용이나 과제 작성 과정에서 질문이 있다면 언제든 **질문**하세요!~
- E-mail:cray25@sasa.hs.kr